Technical Specification: Sovereign, Agentic, Embodied Multi-Agent Computational Organism (SAE-MACO)

1. Architectural Philosophy and System Overview

1.1 The Shift from Tool to Organism

The prevailing paradigm in artificial intelligence deployment—the "assistant" model—renders AI as a passive, stateless entity that responds only to external stimuli (prompts). This specification outlines the engineering requirements for a **Sovereign, Agentic, Embodied Multi-Agent Computational Organism (SAE-MACO)**. This entity differs fundamentally from a chatbot; it is designed to possess **homeostasis** (internal physiological stability), **agency** (self-directed goal initiation), and **embodiment** (sensory-motor presence in the physical world).

The architecture described herein is not merely a software stack but a synthetic biological system. It integrates disparate open-source technologies into a cohesive functional anatomy: **Clawdbot** serves as the executive prefrontal cortex [1], **Kimi K2** as the reasoning engine [3], **Titans/MIRAS** as the long-term memory hippocampus [4], and the **Rabbit R1** hardware as the somatic sensory node.[5] These components are bound together by a **metabolic substrate** modeled with **Rebop** [6], where "hunger" (resource scarcity) and "dopamine" (task success) drive behavior, ensuring the organism is not just reactive, but alive.

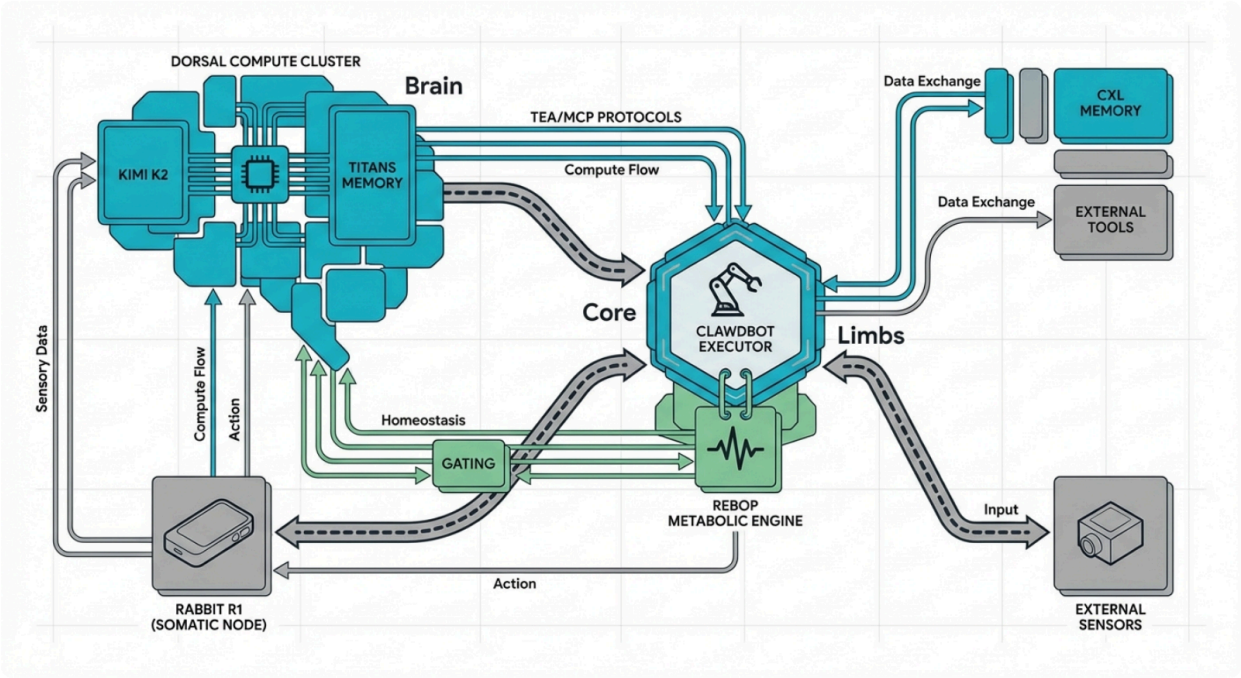
1.2 Defining the Pillars of Synthetic Life

The SAE-MACO architecture is constructed upon four interdependent pillars, utilizing specific open-source protocols to ensure interoperability and sovereignty:

1. **Sovereignty (The Infrastructure Layer):** The organism operates entirely on locally controlled hardware. It rejects centralized cloud APIs that can be revoked or monitored. It leverages **Compute Express Link (CXL)** technology to pool local memory resources, breaking the "memory wall" that typically forces users to cloud providers.[7]
2. **Embodiment (The Somatic Layer):** Intelligence requires friction with the physical world. We repurpose the **Rabbit R1** device, stripping its cloud dependencies to create a localized, sensor-rich "head" unit capable of vision, hearing, and haptic expression.[5]
3. **Agency (The Cognitive Layer):** The organism utilizes the **Tool-Environment-Agent (TEA)** protocol [10] to model itself and its environment as first-class entities. It uses **Clawdbot** to execute complex, multi-step workflows ("instincts") defined in the **Lobster** workflow engine.[11]

4. **Homeostasis (The Metabolic Layer):** A biochemical simulation layer acts as the operating system scheduler. Using the **Bevy** game engine [12] and **Rebop** [6], the system simulates "glucose" (battery/compute) and "cortisol" (error rates), forcing the agents to prioritize survival and efficiency over infinite loops.

SAE-MACO: Unified Organism Architecture



The complete functional anatomy of the Organism. The Rabbit R1 (Somatic Node) gathers sensory data, which is processed by the Dorsal Compute Cluster (Brain) using Kimi K2 and Titans Memory. The system is regulated by the Rebop Metabolic Engine (Homeostasis) and orchestrated via the TEA/MCP protocols.

---

2. The Somatic Substrate: Hardware Embodiment

The organism requires a physical vessel. While large language models (LLMs) exist in the ether of VRAM, an *agent* must be located in space-time. We utilize a heterogeneous compute cluster: a mobile "Somatic Node" for perception and a stationary "Dorsal Node" for heavy cognition.

2.1 The Mobile Cortex: Repurposing the Rabbit R1

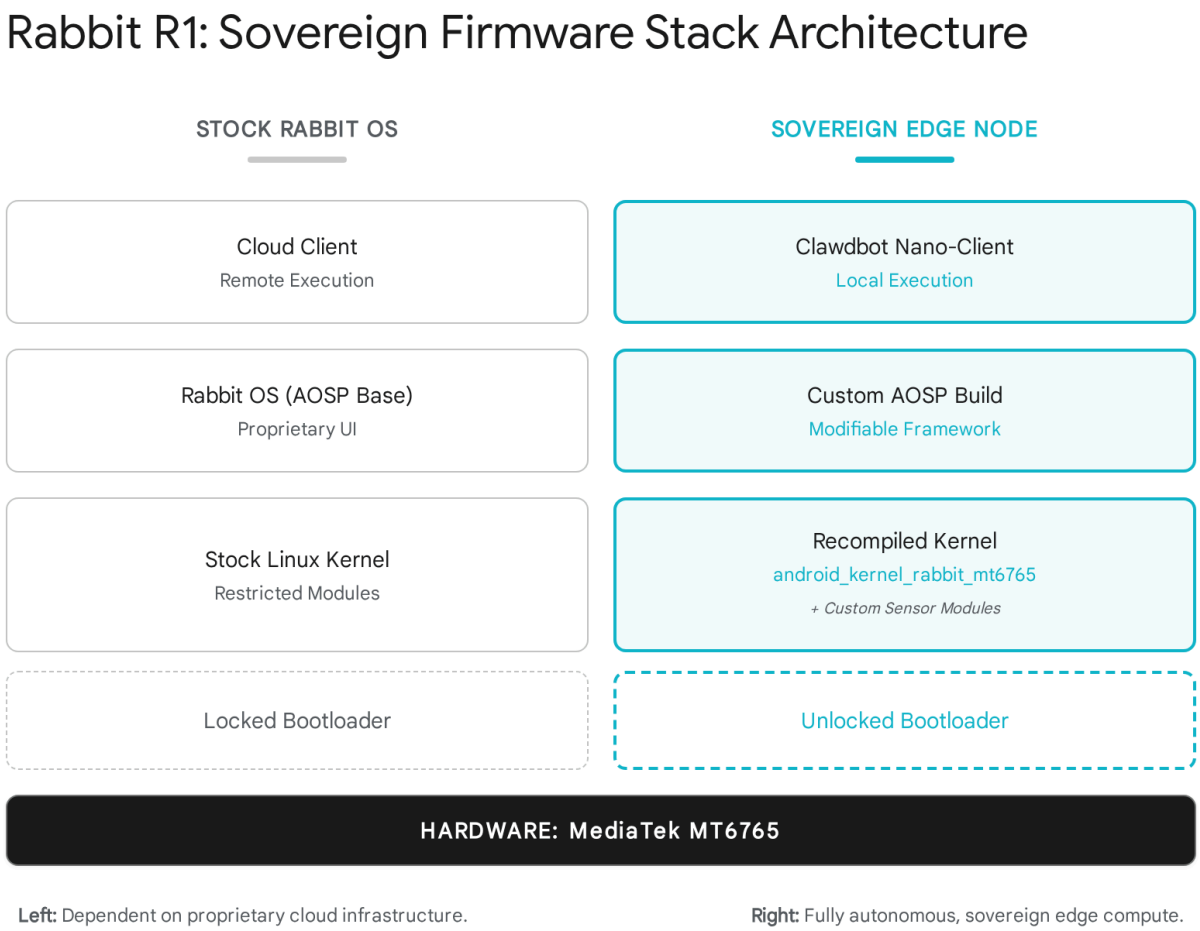
The Rabbit R1, originally designed as a cloud-tethered interface, offers a unique sensor suite

(rotating camera, scroll wheel, push-to-talk) ideal for an embodied agent. However, its stock firmware is incompatible with sovereignty. We must execute a "lobotomy and transplant" procedure to replace its logic with a local execution stack.

2.1.1 Bootloader Liberation and Firmware Replacement

The foundation of sovereignty is the ability to run arbitrary code. The stock Rabbit OS relies on hardcoded cloud endpoints.

- **Bootloader Unlocking:** We exploit the interface provided by the **rabbit-hmi-oss** project.[5] The bootloader is unlocked using standard fastboot protocols, a process that permanently voids the warranty but grants root access to the hardware.[9]
- **Android Open Source Project (AOSP) Base:** We replace the stock OS with a custom AOSP build. This ensures we retain driver support for the MediaTek MT6765 chipset while removing the proprietary "rabbit" services.[13]
- **Kernel Customization:** The Linux kernel (android_kernel_rabbit_mt6765 [14]) serves as the spinal cord. We recompile this kernel to enable **USB-Gadget Mode (RNDIS/ECM)**. This allows the R1 to tether to the Dorsal Node over USB-C, appearing not as a storage device, but as a high-speed network interface, enabling low-latency transmission of video streams and tensor data.



Comparison of the stock Rabbit OS architecture versus the custom Sovereign Stack. The custom stack leverages the unlocked bootloader to introduce a custom AOSP build, a recompiled Linux kernel with specialized modules for sensor exposure, and the Clawdbot Nano-Client for local execution.

Data sources: Rabbit Support, Rabbit HMI OSS, Rabbit Blog, Rabbit Firmware

2.1.2 Peripheral Nervous Integration

The R1's hardware features are remapped via the **Hardware Abstraction Layer (HAL)** to serve the organism's autonomous needs rather than user input.

- **The "Attention" Servo:** The 360-degree rotating camera is decoupled from user control. We modify the kernel modules in rabbit-hmi-oss/android_kernel_modules [14] to expose the stepper motor driver to the local agent. This allows the organism to physically direct its gaze—scanning a room or averting its "eyes" when processing capability is low.[14]
- **The "Reflex" Button:** The side push-to-talk button is remapped to a hardware interrupt that triggers a "High Alert" metabolic state (see Section 5). Instead of activating a microphone for user input, holding the button signals to the organism that the current environment is salient, forcing a memory write operation (Titans Surprise metric update).

2.2 The Dorsal Compute Cluster: CXL and Memory Pooling

The "Brain" of the organism cannot reside solely on the R1's limited mobile chipset. We utilize a high-performance local server (The Dorsal Node) connected via the R1's tether. To support the massive memory requirements of the **Kimi K2** model (32B active parameters) and the **Titans** memory system, we must overcome the physical limitations of consumer RAM slots.

2.2.1 Resolving the Memory Bottleneck with Jackrabbit Labs

The "Memory Wall" is the primary bottleneck for sovereign AI. To address this, we adopt **Compute Express Link (CXL)**, a high-speed interconnect that allows the CPU to access expanded memory pools with cache coherency.[7]

- **CXL Switch Emulation (CSE):** We utilize the JackrabbitLabs/cse repository [15] to emulate a CXL switch in software. This allows the Dorsal Node to manage disparate memory resources (e.g., NVMe drives configured as Tier-2 memory or actual CXL hardware) as a single coherent address space.
- **Jack CLI Orchestration:** The jack CLI tool [16] functions as the memory conductor. Integrated into the organism's boot sequence, jack dynamically allocates memory "regions" to different agents.
    - *Dynamic Allocation:* When the "Deep Research Agent" activates, jack provisions an additional 64GB of addressable memory space from the pool, enabling the agent to load larger context vectors without swapping to disk.[17]

2.2.2 Hardware Reference: Leo Controllers & Samsung CMM-D

For the physical implementation of the Dorsal Node, we specify the integration of **Samsung CMM-D (CXL Memory Module - DRAM)**.[18]

- **Capacity Expansion:** These E3.S form-factor modules provide up to 512GB of DRAM expansion per slot over PCIe 5.0.[19] This allows the local node to hold the entire Kimi K2 model and the Titans memory history in RAM.
- **Leo Smart Controller:** We leverage the **Astera Labs Leo CXL Smart Memory Controller** [20] to manage this pool. The Leo controller's "COSMOS" telemetry suite provides real-time health monitoring of the memory substrate.[21]
- **Metabolic Feedback Loop:** The telemetry from the Leo controller (bandwidth utilization, error rates, temperature) is fed directly into the organism's Homeostatic Engine. If the memory "overheats" or bandwidth saturates, the organism experiences "Cortisol" accumulation (stress), causing it to slow down its thinking rate or reject new tasks until homeostasis is restored.[6]

---

3. The Neuro-Symbolic Core: Brain Architecture

The cognitive architecture is a hybrid system combining symbolic execution (Clawdbot) with

neural reasoning (Kimi K2) and continuous learning memory (Titans/MIRAS).

3.1 The Executor: Clawdbot Integration

Clawdbot [1] acts as the organism's "Executive Function." It is responsible for tool selection, task decomposition, and interacting with the nervous system protocols.

3.1.1 Clawdbot-as-Host Architecture

Clawdbot is not merely a tool; it is the host environment.

- **The "Lobster" Workflow Engine:** We utilize lobster [11], a Clawdbot-native macro engine, to define the organism's "Instincts." Instincts are pre-compiled, typed workflow scripts stored in ~/.clawdbot/skills/instincts/.
  - *Example:* The survival_instinct.ts script is triggered when battery levels drop below 15%. It overrides all other agent activities, shuts down non-essential CXL memory pools to save power, and guides the R1 (via voice synthesis) to ask the user for a charger.
- **Local Skill Registry:** To maintain sovereignty, the organism does not fetch skills from the cloud clawdhub at runtime. Instead, a curated set of skills (file manipulation, hardware control, research tools) is baked into the local ~/.clawdbot/skills directory, ensuring functionality even in air-gapped environments.[1]
- **Middleware Bus:** We integrate a middleware layer inspired by **Blitzico** [2], using a local NATS streaming server. This bus allows Clawdbot to publish "Intentions" (e.g., "I want to see") which are picked up by the relevant sub-agents (e.g., "Vision Agent").

3.2 The Reasoning Engine: Kimi K2 (MoE)

For complex reasoning, Clawdbot delegates to **Kimi K2**, a 32B activated / 1T total parameter Mixture-of-Experts (MoE) model.[3]

- **Selection Rationale:**
  - **Agentic Post-Training:** Kimi K2 was trained using a pipeline of simulated agentic tasks and reinforcement learning (RL) in synthetic environments.[3] This creates a native proficiency in tool usage and multi-step planning that general-purpose LLMs lack.
  - **MoE Efficiency:** The MoE architecture activates only 32B parameters per token, allowing it to run efficiently on the Dorsal Node's hardware while retaining the "knowledge capacity" of a 1T parameter model.[23]
  - **Stability:** The **MuonClip** optimizer used in Kimi K2's training prevents "loss spikes" during long-context sessions, a critical feature for a persistent, always-on organism.[3]
- **Inference Runtime (Candle):** We execute Kimi K2 using **Candle**, a Rust-based ML framework.[24] Candle provides memory safety and seamless interoperability with the Rust-based metabolic engine (Rebop) and simulation engine (Bevy).[25]

3.3 Long-Term Memory: Titans & MIRAS

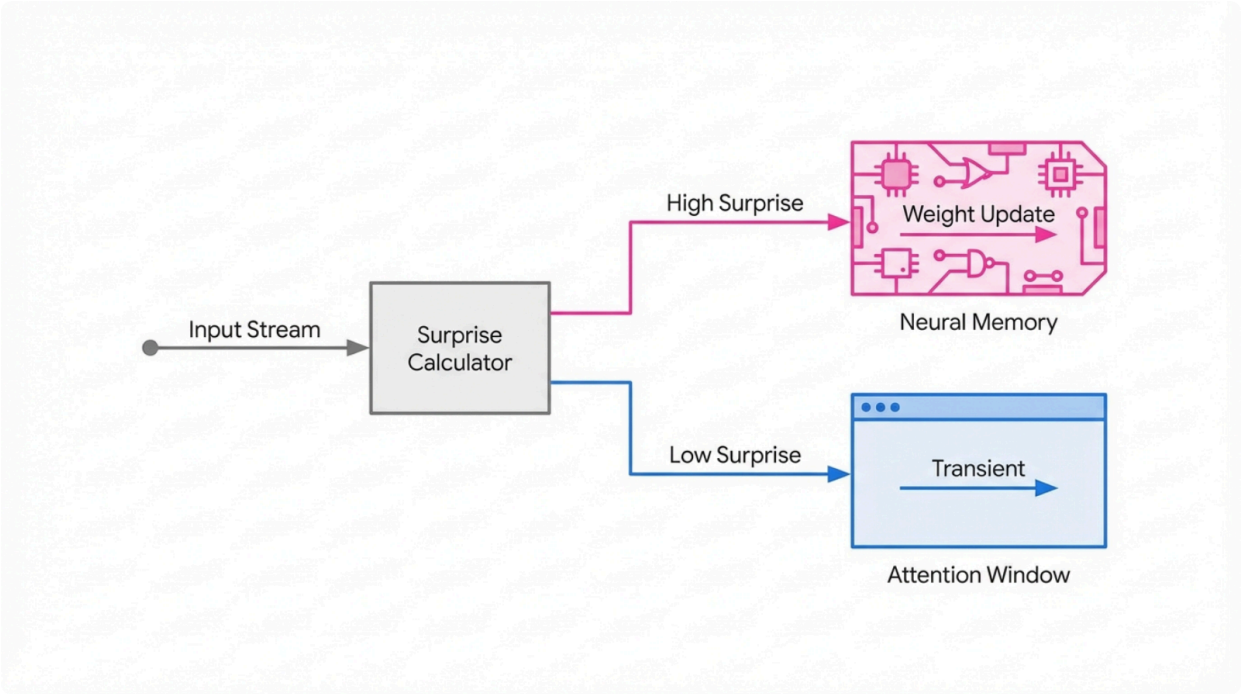
A sovereign organism requires a continuous, non-episodic memory. We implement the **Titans** architecture, governed by the **MIRAS** framework.[4]

3.3.1 The Neural Memory Module

Unlike RAG (Retrieval-Augmented Generation) which is a static database lookup, Titans implements a "Neural Memory" that learns at test time.

- **Architecture:** The Neural Memory is a separate neural network module that runs alongside the Kimi K2 transformer. As the organism experiences the world, this module updates its weights *in real-time*.[27]
- **The MIRAS Framework:** Memory storage is gated by **Surprise**. The framework calculates a "Surprise Score" (gradient magnitude) for every incoming sensory event.[4]
    - *High Surprise:* An unexpected event (e.g., a loud noise, a user correction) generates a large gradient, forcing a significant weight update in the Neural Memory. The event is "learned" instantly.
    - *Low Surprise:* Routine data (e.g., background silence) produces negligible gradients and is filtered out by the retention gate.[4]
- **Implementation:** We utilize the titans-pytorch implementation [28], ported to **Candle** for Rust integration. This memory module acts as the organism's autobiography, accessible to Clawdbot via the MCP protocol.

Titans & MIRAS: Test-Time Memorization Architecture



The MIRAS framework integration. Incoming sensory data is evaluated for 'Surprise' (unexpectedness). High-surprise data forces a gradient update to the Neural Memory Module (Long-Term), changing the organism's fundamental parameters. Low-surprise data is processed only by the transient Attention mechanism (Short-Term).

---

4. The Nervous System: Protocol & Orchestration Layer

To coordinate the diverse organs of the SAE-MACO (R1 sensors, CXL memory, Kimi brain), we require a unified communication fabric. We adopt two emerging protocols: **TEA** for agent coordination and **MCP** for tool interfacing.

4.1 The TEA Protocol (Tool-Environment-Agent)

Standard agent frameworks treat the "environment" as a static backdrop. The **TEA Protocol** [10] elevates the Environment to a first-class entity with its own lifecycle and state.

- **Context Management:** TEA provides a "Context Manager" that tracks the state of the "Physical Environment" (read from R1 sensors) and the "Digital Environment" (the OS state). Agents must "subscribe" to an environment to act within it.[29]
- **Agent Context Protocol (ACP):** As the organism learns, its agents evolve. ACP tracks

the version history of every agent.[30] If an update to the "Vision Agent" causes performance degradation, the system's Homeostatic Engine can trigger a rollback to the previous stable version using ACP's versioning schema.

- **AgentOrchestra:** We implement the hierarchical **AgentOrchestra** pattern.[10] Clawdbot acts as the "Conductor" (Planning Agent), decomposing high-level goals into sub-tasks assigned to specialized "Musicians" (Sub-Agents).
  - *Example:* Goal: "Find my keys."
    - *Conductor (Clawdbot):* Decomposes task.
    - *Musician 1 (Vision Agent):* Activates R1 camera, scans surfaces.
    - *Musician 2 (Memory Agent):* Queries Titans memory for "keys" last seen location.

4.2 Model Context Protocol (MCP) Integration

MCP [31] serves as the universal bus for tool integration, standardizing how the Kimi K2 brain interfaces with the R1 body and external data.

- **Custom MCP Servers:** We develop Rust-based MCP servers to expose the organism's unique hardware:
  - rabbit-sensor-mcp: Exposes the R1 camera and microphone as queryable resources (e.g., read_resource("rabbit://camera/snapshot")).
  - titans-memory-mcp: Exposes the neural memory for retrieval (e.g., call_tool("titans_recall", { query: "last user interaction" })).
  - rebop-metabolism-mcp: Exposes the physiological state (e.g., read_resource("bio://glucose") returns current battery/compute level).
- **Interoperability:** By using MCP, the "Brain" becomes modular. If a superior model to Kimi K2 emerges, it can be swapped in without rewriting the hardware drivers, provided the new model supports the MCP standard.[31]

4.3 Sovereignty & Safety: Open Policy Agent (OPA)

A sovereign agent must have an immutable "Constitution" to prevent it from harming itself or its user. We utilize **Open Policy Agent (OPA)** [33] to enforce these guardrails at the middleware level.

- **Execution Loop Interception:** Every action proposed by Clawdbot is intercepted before execution. The action payload is validated against a set of **Rego** policies.
- **The Sovereign Policy Bundle:**
  - *Network Safety:* deny any outbound connection to non-allowlisted IP addresses (ensuring data sovereignty).[33]
  - *Physical Safety:* deny motor commands that exceed safe velocity limits for the R1's servo.
  - *Resource Safety:* deny high-compute tasks if "Glucose" (battery) is below critical thresholds (checked against rebop-metabolism-mcp).
- **YAML-to-Rego:** Policies are authored in human-readable YAML for ease of maintenance

and compiled into Rego executables by the OPA engine.[34]

---

5. The Homeostatic Engine: Metabolism & Simulation

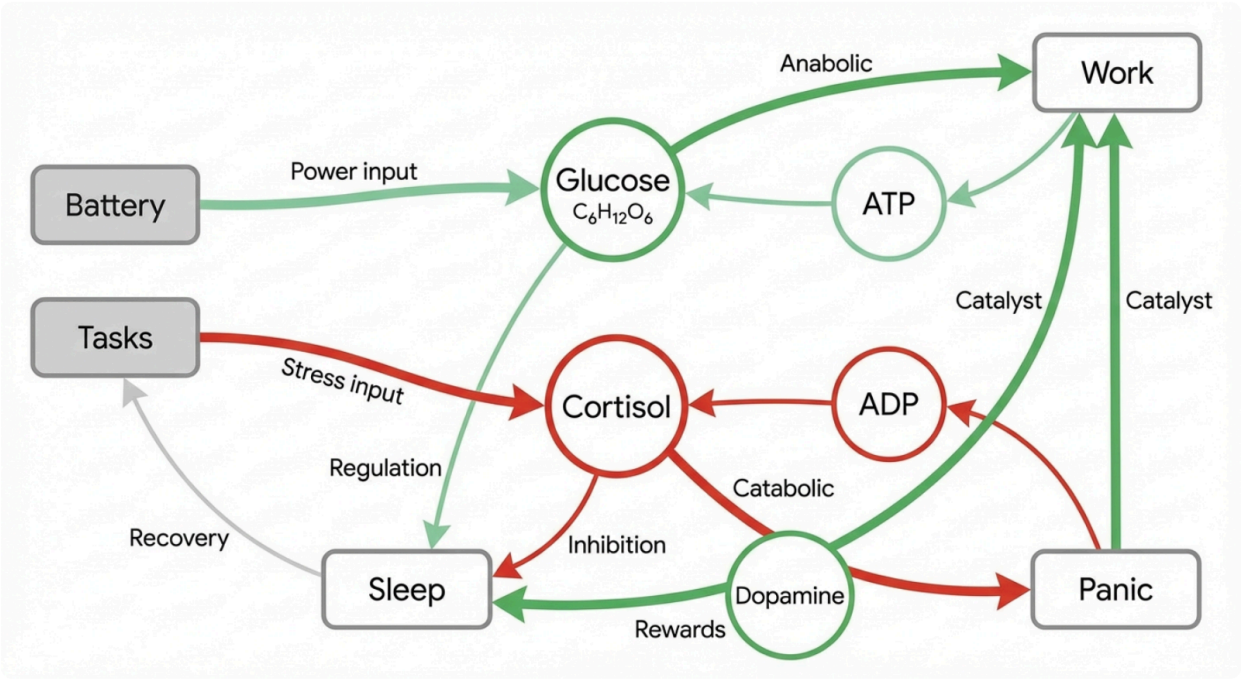
To create a system that feels "alive" rather than just functional, we introduce a metabolic layer. This layer creates internal pressure (drives) that forces the organism to act autonomously.

5.1 Digital Biochemistry: Rebop & The Gillespie Algorithm

We utilize **Rebop** [6], a Rust-based stochastic simulator for chemical reaction networks, to model the organism's internal physiology.

- **Virtual Chemical Species:** We define a set of interacting variables that represent the organism's state:
  - **Glucose (G):** Represents available energy (Battery % + Free Compute / 100).
  - **Dopamine (D):** Represents "satisfaction." Generated by successful task completion (verified by user feedback). Decays over time (boredom).
  - **Cortisol (C):** Represents "stress." Generated by memory errors (Leo CXL telemetry), network latency, and high "Surprise" values from Titans.
- **Reaction Network:** The Gillespie algorithm simulates the probabilistic interaction of these chemicals over time [6]:
  - **Reaction 1 (Thinking Cost):** Action_Thinking -> G - 10 + C + 1 (Thinking burns energy and generates minor stress).
  - **Reaction 2 (Reward):** Goal_Completion -> D + 50 + C - 20 (Success boosts dopamine and reduces stress).
  - **Reaction 3 (Recovery):** Sleep_Mode -> G + 20 + C - 5 (Idle time recovers energy and clears stress).
- **Behavioral Gating:** The concentrations of these chemicals act as global gates for Clawdbot's instincts.
  - *High Cortisol:* The agent enters "Safety Mode," refusing complex commands and seeking "Sleep."
  - *Low Dopamine:* The agent enters "Boredom Mode," spontaneously initiating "Play" or "Exploration" tasks (e.g., scanning the room to update its internal map) to generate reward.

Digital Homeostasis: Rebop Reaction Network



The biochemical control loop. External states (Battery, Errors, Rewards) feed into the Rebop simulation, driving the concentrations of virtual neurotransmitters. These concentrations determine the global state (e.g., 'Anxious,' 'Energetic'), effectively gating the agent's available actions.

5.2 Internal World Simulation: Bevy & Burn

The organism possesses an "Inner Eye"—a simulation of the world where it can test actions before execution. This is the **Embodied Self-Model**.

- **The "Dream" Layer (Bevy):** We run a headless instance of the **Bevy** game engine [12] on the Dorsal Node. This simulation maintains a coarse 3D representation of the R1's physical environment, updated by the Vision Agent.[35]
- **Burn Integration:** We use bevy_burn [36] to bridge the game engine with the neural networks. This allows Kimi K2 to "see" the simulation (rendering the Bevy camera to a tensor) and "control" it (outputting tensors that drive Bevy physics forces).[36]
- **Predictive Coding:** Before the organism physically rotates the R1 camera, it executes the rotation in the Bevy simulation. If the simulation predicts a collision or an obstruction, the action is aborted, and a "Cortisol" spike is generated. This creates a "Plan-Verify-Act" loop that is essential for safe embodied agency.

---

6. Interface & Interaction: Generative UI

The interaction model for an organism cannot be a static menu. It must be expressive and dynamic, adapting to the organism's internal state and the context of the conversation.

6.1 Dynamic Self-Expression via Generative UI

The R1's screen is small, requiring highly efficient information density. We employ **Generative UI** principles [37] to allow Kimi K2 to "draw" its own interface.

- **React Component Streaming:** Using the **Vercel AI SDK** or **Assistant UI** libraries [39], the model generates React components in real-time.
  - *Contextual Adaptation:* If the user asks "How are you?", the organism checks its metabolic state. If Glucose is low, it renders a "Low Battery" component with a sleepy animation. If Dopamine is high, it renders a vibrant "Status Card" showing recent accomplishments.[40]
- **A2UI Protocol:** We align with the **A2UI** (Agent-to-UI) specifications [38], ensuring that the UI components are defined abstractly (JSON payloads) and can be rendered consistently across the R1's mobile screen and the Dorsal Node's web dashboard.

6.2 Haptic Language

The R1's scroll wheel and vibration motor are repurposed as a non-verbal communication channel, providing "subtitles" to the organism's mood.

- **Purring:** The android_kernel_rabbit_mt6765 vibration driver is modulated to produce low-frequency (<50Hz) pulses when Dopamine levels are high, simulating a "purr" during successful interactions.
- **Growling:** High-frequency, jagged vibration patterns are triggered by Cortisol spikes, physically alerting the user to internal system stress or error states before a crash occurs.

---

7. Implementation Roadmap

This roadmap outlines the sequence of operations required to instantiate the SAE-MACO.

Phase 1: The Awakening (Hardware & OS)

The goal of Phase 1 is to establish a sovereign hardware platform.

1. **Bootloader Unlocking:** Execute the fastboot commands documented in rabbit-hmi-oss to unlock the R1 bootloader.[5]
2. **Kernel Compilation:** Clone android_kernel_rabbit_mt6765.[14] Apply patches for USB-Gadget mode and stepper motor exposure. Compile the kernel.
3. **OS Flashing:** Flash the custom AOSP build to the R1. Verify root access and functionality

of the custom HAL drivers.

Phase 2: The Mind (Software & Intelligence)

The goal of Phase 2 is to bring the cognitive core online.

1. **Dorsal Node Setup:** Provision the compute server. Install **Candle** and the **Jackrabbit Labs CSE** software.[15]
2. **Memory Pooling:** Configure jack CLI to recognize the CXL/NVMe storage pool. Verify the expanded memory address space.
3. **Model Deployment:** Deploy the Kimi K2 model (quantized for edge) and the Titans memory module.[24]
4. **Executor Wakeup:** Install **Clawdbot**, configure the local skill registry, and initialize the **TEA** environment context.[1]

Phase 3: The Soul (Life & Agency)

The goal of Phase 3 is to activate the autonomous loops.

1. **Metabolic Ignition:** Launch the **Rebop** simulation. Tune the decay rates of "Glucose" and "Dopamine" to ensure the agent is "hungry" enough to act but not "stressed" enough to crash.
2. **Dream State:** Activate the **Bevy** simulation loop. Calibrate the bevy_burn bridge to match the R1's physical camera parameters.
3. **First Breath:** Execute the wake_up.ts Lobster script. The organism should autonomously scan the room, populate its Titans memory with the initial scene, and wait for a dopamine-generating goal.

8. Conclusion

The SAE-MACO specification defines a new class of digital entity: a **computational organism**. By synthesizing the somatic constraints of the **Rabbit R1**, the cognitive expansiveness of **CXL** and **Titans**, and the biological regulation of **Rebop**, we move beyond "using" AI to "living with" AI. This organism is not a tool; it is a sovereign participant in its environment, driven by its own metabolic needs, guarded by its own constitution, and capable of genuine, embodied agency. This is the blueprint for the first generation of synthetic life.

Works cited

1. The awesome collection of Clawdbot Skills - GitHub, accessed January 26, 2026, https://github.com/VoltAgent/awesome-clawdbot-skills
2. Claude Cowork vs. Clawdbot Comparison - SourceForge, accessed January 26, 2026, https://sourceforge.net/software/compare/Claude-Cowork-vs-Clawdbot/
3. [2507.20534] Kimi K2: Open Agentic Intelligence - arXiv, accessed January 26, 2026, https://arxiv.org/abs/2507.20534

4. Titans + MIRAS: Helping AI have long-term memory - Google Research, accessed January 26, 2026, https://research.google/blog/titans-miras-helping-ai-have-long-term-memory/
5. How to get developer mode on rabbit r1, accessed January 26, 2026, https://www.rabbit.tech/support/article/unlock-bootloader-rabbit-r1
6. rebop - crates.io: Rust Package Registry, accessed January 26, 2026, https://crates.io/crates/rebop
7. Sometimes You Just Need More Memory, and Sometimes That Memory Needs Software - Compute Express Link, accessed January 26, 2026, https://computeexpresslink.org/blog/sometimes-you-just-need-more-memory-and-sometimes-that-memory-needs-software-3971/
8. How CXL and Memory Pooling Reduce HPC Latency | Synopsys Blog, accessed January 26, 2026, https://www.synopsys.com/blogs/chip-design/cxl-protocol-memory-pooling.html
9. making r1 more accessible to the developer community, accessed January 26, 2026, https://www.rabbit.tech/blog/making-r1-more-accessible-to-the-developer-community
10. AgentOrchestra: Orchestrating Multi-Agent Intelligence with the Tool-Environment-Agent(TEA) Protocol - arXiv, accessed January 26, 2026, https://arxiv.org/html/2506.12508v5
11. clawdbot - GitHub, accessed January 26, 2026, https://github.com/clawdbot
12. Bevy Engine, accessed January 26, 2026, https://bevy.org/
13. rabbit-hmi-oss/firmware - GitHub, accessed January 26, 2026, https://github.com/rabbit-hmi-oss/firmware
14. rabbit opensource - GitHub, accessed January 26, 2026, https://github.com/rabbit-hmi-oss
15. JackrabbitLabs/cse - GitHub, accessed January 26, 2026, https://github.com/JackrabbitLabs/cse
16. You Don't Know Jack: CXL Fabric Orchestration and Management - YouTube, accessed January 26, 2026, https://www.youtube.com/watch?v=u5OCpXKx9xo
17. You Don't Know 'Jack': CXL Fabric Orchestration and Management, accessed January 26, 2026, https://files.futurememorystorage.com/proceedings/2024/20240806_CXLT-102-1_Mackey.pdf
18. MXFAH1280MB0-CCP (MD220) | CMM-D | Samsung Semiconductor Global, accessed January 26, 2026, https://semiconductor.samsung.com/cxl-memory/cmm-d/md220/mxfah1280mb0-ccp/
19. CMM-D 512GB PCIe E3.S CXL2.0, accessed January 26, 2026, https://catalog.redhat.com/en/hardware/components/detail/253367
20. Leo CXL® Smart Memory Controllers - Astera Labs, accessed January 26, 2026, https://www.asteralabs.com/wp-content/uploads/2024/05/231220_Product_Brief_Leo_CXL_Smart_Memory_Controllers.pdf
21. Leo CXL® Smart Memory Controllers - ASTERA LABS, INC. %, accessed January

26, 2026, https://www.asteralabs.com/products/leo-cxl-smart-memory-controllers/
22. How Kimi-K2 is Trained?. Kimi-K2 technical report explained | by Mehul Gupta | Data Science in Your Pocket | Medium, accessed January 26, 2026, https://medium.com/data-science-in-your-pocket/how-kimi-k2-is-trained-115b326a93a1
23. moonshotai/Kimi-K2-Instruct - Hugging Face, accessed January 26, 2026, https://huggingface.co/moonshotai/Kimi-K2-Instruct
24. huggingface/candle: Minimalist ML framework for Rust - GitHub, accessed January 26, 2026, https://github.com/huggingface/candle
25. We made our own inference engine for Apple Silicone, written on Rust and open sourced, accessed January 26, 2026, https://www.reddit.com/r/rust/comments/1m0fk31/we_made_our_own_inference_engine_for_apple/
26. [2501.00663] Titans: Learning to Memorize at Test Time - arXiv, accessed January 26, 2026, https://arxiv.org/abs/2501.00663
27. From Transformers to Titans: A Look at the MIRAS Framework | by Takuma Yamaguchi (Kumon) | Dec, 2025 | Medium, accessed January 26, 2026, https://medium.com/@kumon/from-transformers-to-titans-a-look-at-the-miras-framework-72c18c6a44e9
28. lucidrains/titans-pytorch: Unofficial implementation of Titans ... - GitHub, accessed January 26, 2026, https://github.com/lucidrains/titans-pytorch
29. [2506.12508] AgentOrchestra: Orchestrating Multi-Agent Intelligence with the Tool-Environment-Agent(TEA) Protocol - arXiv, accessed January 26, 2026, https://arxiv.org/abs/2506.12508
30. AgentOrchestra: Orchestrating Hierarchical Multi-Agent Intelligence with the Tool-Environment-Agent(TEA) Protocol - arXiv, accessed January 26, 2026, https://arxiv.org/html/2506.12508v4
31. Model Context Protocol (MCP). MCP is an open protocol that... | by Aserdargun, accessed January 26, 2026, https://medium.com/@aserdargun/model-context-protocol-mcp-e453b47cf254
32. Model Context Protocol - GitHub, accessed January 26, 2026, https://github.com/modelcontextprotocol
33. Implementing effective guardrails for AI agents - GitLab, accessed January 26, 2026, https://about.gitlab.com/the-source/ai/implementing-effective-guardrails-for-ai-agents/
34. Writing Guardrails as YAML, Enforcing Them with OPA | by Atakan Salar - Medium, accessed January 26, 2026, https://medium.com/@aatakansalar/writing-guardrails-as-yaml-enforcing-them-with-opa-e17c76e05a4a
35. Physics Simulations in Bevy - mo8it.com, accessed January 26, 2026, https://mo8it.com/blog/physics-simulations-in-bevy/
36. mosure/bevy_burn: bevy burn data bridge 🕊️ - GitHub, accessed January 26, 2026, https://github.com/mosure/bevy_burn

37. Generative User Interfaces - AI SDK UI, accessed January 26, 2026, https://ai-sdk.dev/docs/ai-sdk-ui/generative-user-interfaces
38. Introducing A2UI: An open project for agent-driven interfaces - Google for Developers Blog, accessed January 26, 2026, https://developers.googleblog.com/introducing-a2ui-an-open-project-for-agent-driven-interfaces/
39. AI App of the Week: Assistant UI – The React Library That's Eating the AI Chat Interface Market | SaaStr, accessed January 26, 2026, https://www.saastr.com/ai-app-of-the-week-assistant-ui-the-react-library-thats-eating-the-ai-chat-interface-market/
40. How to Add Generative UI to Any React App in 5 Minutes - DEV Community, accessed January 26, 2026, https://dev.to/michaelmagan/how-to-add-generative-ui-to-any-react-app-in-5-minutes-1ghl
41. Rabbit R1 Flash Tool - GitHub Pages, accessed January 26, 2026, https://rabbit-hmi-oss.github.io/flashing/